Firewall Exploration SEED Lab

Matthew Blanchard

CYEN 3331-01

4/15/26

Executive Summary:

This report details the implementation and evaluation of various firewall configurations using the Linux kernel's Netfilter framework and the iptables utility. The project progresses from building a custom loadable kernel module for basic packet filtering to configuring complex stateful rules for network protection. Key findings demonstrate the effectiveness of Netfilter hooks in intercepting traffic and the utility of connection tracking in maintaining secure communication channels. Furthermore, the study explores advanced firewall applications including traffic rate limiting to mitigate network congestion and the use of the statistic module for server load balancing. Ultimately, the results confirm that a properly orchestrated firewall is essential for defending internal network assets against unauthorized external access.

Introduction:

Firewalls serve as the primary line of defense in network security by enforcing access control policies based on predefined rule sets. This lab utilizes the SEED Ubuntu environment and Docker containers to simulate a realistic network topology consisting of an attacker, a router, and multiple internal hosts. By leveraging Netfilter—a framework within the Linux kernel—users can intercept and manipulate packets at various stages of the networking stack. The transition from stateless filtering to stateful inspection allows for a more nuanced understanding of network traffic context, which is critical for modern cybersecurity. Through hands-on tasks, this exploration provides a technical foundation for implementing both fundamental and sophisticated network security measures.

Lab Environment Setup

1. Download and unzip Labsetup.zip, then navigate into the Labsetup directory. Build the containers by running docker-compose build.
2. Start the containers using docker-compose up and stop them when needed with docker-compose down.
3. List running containers by executing docker ps. Open a shell in a container using docker exec -it <container_id> /bin/bash.

Task 1

Objective:

The objective is to understand how firewalls function within the Linux kernel by developing a custom Loadable Kernel Module (LKM). This task focuses on utilizing the Netfilter framework to inspect, filter, and block network packets based on hardcoded security policies.

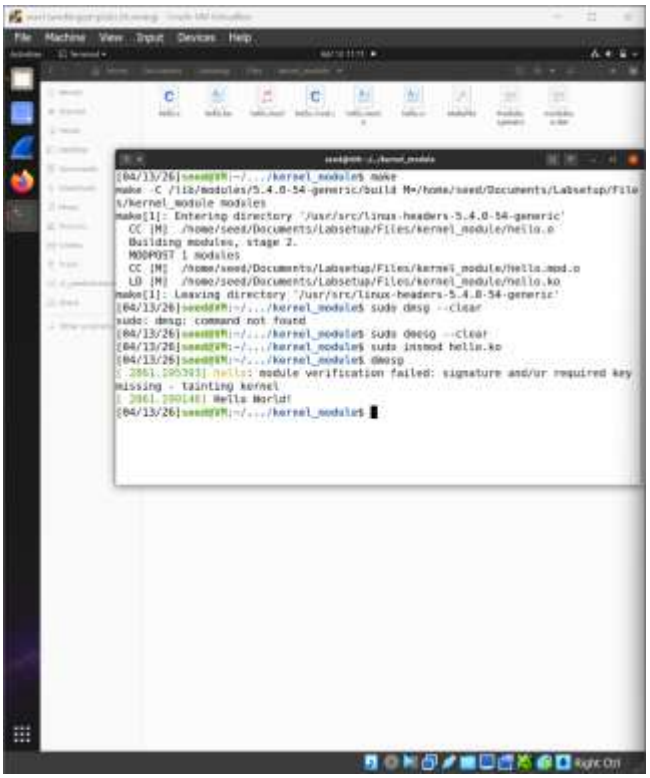
Task 1A

Instructions:

1. Navigate to the directory containing hello.c and the Makefile, then compile the module using make.

2. Insert the module with sudo insmod hello.ko and verify it using lsmod | grep hello.
3. Check kernel messages using dmesg to confirm the module executed.
4. Remove the module using sudo rmmod hello and confirm removal by running dmesg again.

Observations:



Running dmesg after compiling and loading the hello module into the kernel displays the correct “Hello World!” message at the end of the dmesg dump.

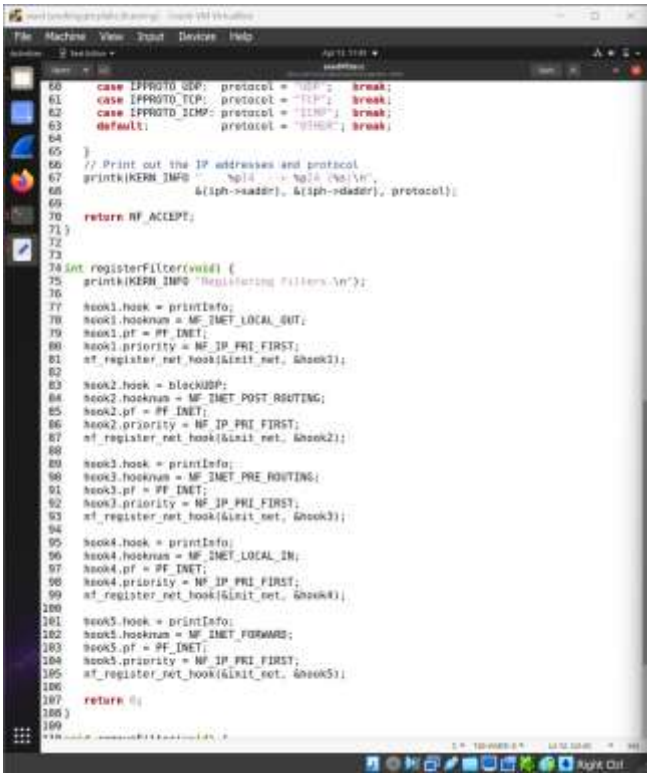
Task 1B

Instructions:

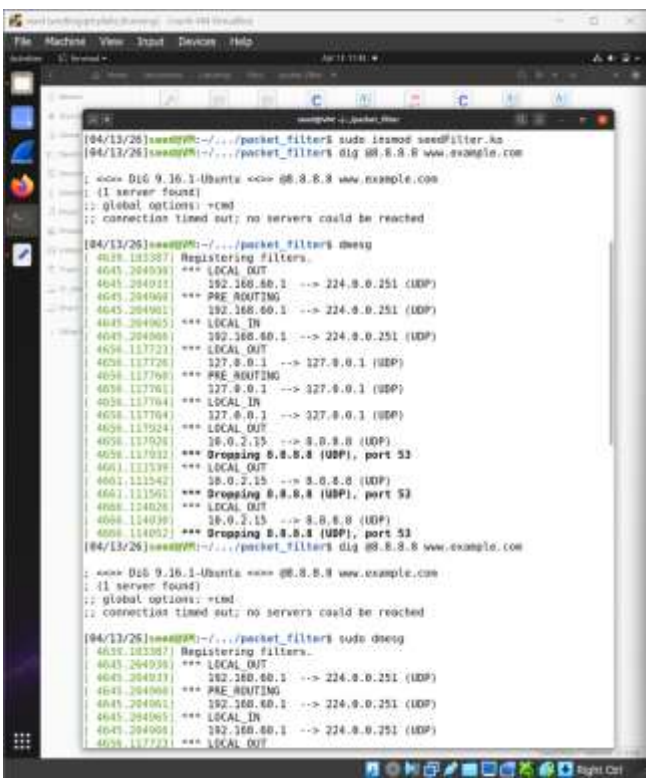
1. Navigate to the packet_filter directory and compile the firewall code using make.
2. Load the module with sudo insmod seedFilter.ko and test UDP blocking using dig @8.8.8.8 www.example.com.
3. Modify the code to register hooks for PRE_ROUTING, LOCAL_IN, FORWARD, LOCAL_OUT, and POST_ROUTING, then recompile and reload the module.
4. Generate network traffic and observe hook execution using dmesg logs.
5. Implement a hook function to block ICMP echo requests and another to block TCP port 23 traffic.
6. Register both hooks to an appropriate Netfilter hook, then recompile and reload the module.
7. Test the configuration by running ping <VM_IP> and telnet <VM_IP> from another container and verify both are blocked.

8. Ensure all hooks are unregistered in the module exit function. Remove the module safely using sudo rmmod seedFilter.

Observations:



```
    case IPPROTO_UDP:  protocol = "UDP";  break;
    case IPPROTO_TCP:  protocol = "TCP";  break;
    case IPPROTO_ICMP: protocol = "ICMP"; break;
    default:           protocol = "OTHER"; break;

  }
  // Print out the IP addresses and protocol
  printk(KERN_INFO "    %pI4  -> %pI4 (%d)\n",
                 &(iph->saddr), &(iph->daddr), protocol);

  return NF_ACCEPT;
}


int registerFilter(void) {
  printk(KERN_INFO "Registering filters.\n");

  hook1.hook = printInfo;
  hook1.hooknum = NF_INET_LOCAL_OUT;
  hook1.pf = PF_INET;
  hook1.priority = NF_IP_PRI_FIRST;
  nf_register_net_hook(&init_net, &hook1);

  hook2.hook = blockUDP;
  hook2.hooknum = NF_INET_POST_ROUTING;
  hook2.pf = PF_INET;
  hook2.priority = NF_IP_PRI_FIRST;
  nf_register_net_hook(&init_net, &hook2);

  hook3.hook = printInfo;
  hook3.hooknum = NF_INET_PRE_ROUTING;
  hook3.pf = PF_INET;
  hook3.priority = NF_IP_PRI_FIRST;
  nf_register_net_hook(&init_net, &hook3);

  hook4.hook = printInfo;
  hook4.hooknum = NF_INET_LOCAL_IN;
  hook4.pf = PF_INET;
  hook4.priority = NF_IP_PRI_FIRST;
  nf_register_net_hook(&init_net, &hook4);

  hook5.hook = printInfo;
  hook5.hooknum = NF_INET_FORWARD;
  hook5.pf = PF_INET;
  hook5.priority = NF_IP_PRI_FIRST;
  nf_register_net_hook(&init_net, &hook5);

  return 0;
}
```



```
[04/13/26]seed@VM:~/.../packet_filter$ sudo insmod seedFilter.ko
[04/13/26]seed@VM:~/.../packet_filter$ dig @8.8.8.8 www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> @8.8.8.8 www.example.com
; (1 server found)
;; global options: +cmd
;; connection timed out; no servers could be reached

[04/13/26]seed@VM:~/.../packet_filter$ dmesg
[ 4639.183387] Registering filters.
[ 4645.204938] *** LOCAL_OUT
[ 4645.204933]     192.168.60.1  --> 224.0.0.251 (UDP)
[ 4645.204968] *** PRE_ROUTING
[ 4645.204961]     192.168.60.1  --> 224.0.0.251 (UDP)
[ 4645.204965] *** LOCAL_IN
[ 4645.204968]     192.168.60.1  --> 224.0.0.251 (UDP)
[ 4656.117723] *** LOCAL_OUT
[ 4656.117726]     127.0.0.1  --> 127.0.0.1 (UDP)
[ 4656.117760] *** PRE_ROUTING
[ 4656.117761]     127.0.0.1  --> 127.0.0.1 (UDP)
[ 4656.117764] *** LOCAL_IN
[ 4656.117764]     127.0.0.1  --> 127.0.0.1 (UDP)
[ 4656.117924] *** LOCAL_OUT
[ 4656.117928]     10.0.2.15  --> 8.8.8.8 (UDP)
[ 4656.117932] *** Dropping 8.8.8.8 (UDP), port 53
[ 4661.111539] *** LOCAL_OUT
[ 4661.111542]     10.0.2.15  --> 8.8.8.8 (UDP)
[ 4661.111561] *** Dropping 8.8.8.8 (UDP), port 53
[ 4666.114026] *** LOCAL_OUT
[ 4666.114030]     10.0.2.15  --> 8.8.8.8 (UDP)
[ 4666.114052] *** Dropping 8.8.8.8 (UDP), port 53
[04/13/26]seed@VM:~/.../packet_filter$ dig @8.8.8.8 www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> @8.8.8.8 www.example.com
; (1 server found)
;; global options: +cmd
;; connection timed out; no servers could be reached

[04/13/26]seed@VM:~/.../packet_filter$ sudo dmesg
[ 4639.183387] Registering filters.
[ 4645.204938] *** LOCAL_OUT
[ 4645.204933]     192.168.60.1  --> 224.0.0.251 (UDP)
[ 4645.204968] *** PRE_ROUTING
[ 4645.204961]     192.168.60.1  --> 224.0.0.251 (UDP)
[ 4645.204965] *** LOCAL_IN
[ 4645.204968]     192.168.60.1  --> 224.0.0.251 (UDP)
[ 4656.117723] *** LOCAL_OUT
```

For the first part, after loading the seedFilter module into the kernel, 8.8.8.8 is blocked and dmesg shows the module actively dropping the IP. Then, after modifying the source code of the module and reloading it back into the kernel, again, 8.8.8.8 is blocked, but this time, the

other filters are shown and displays their relevant information. Further explanation of each of the new filters is as follows:

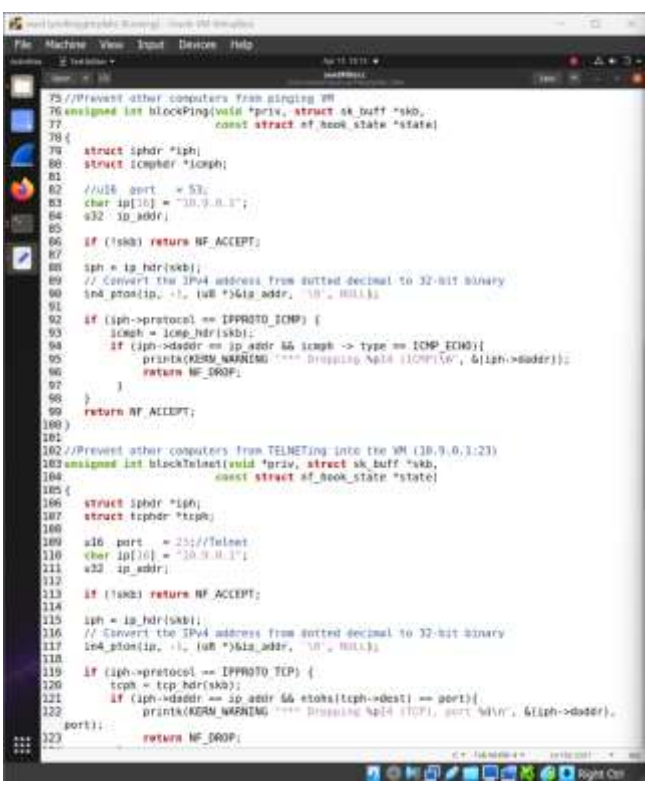
1. NF_INET_PRE_ROUTING happens when a packet is received by the network stack, before the routing decision is made. When incoming traffic (like ping or external requests) was sent to the machine, logs appeared at this stage first. This confirms that packets are intercepted immediately after arriving at the network interface.

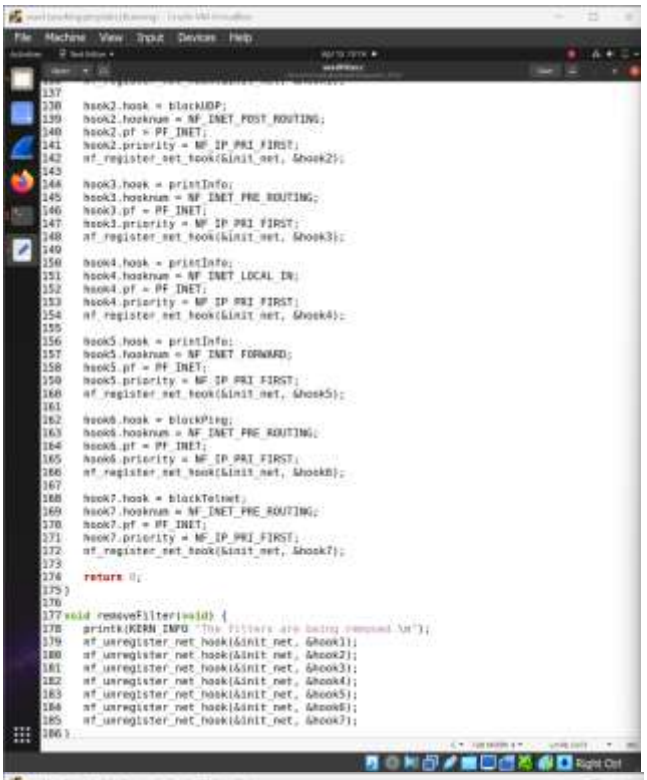
2. NF_INET_LOCAL_IN happens when a packet is about to be delivered locally (the packet's destination IP address matches one of the local system's interfaces). It happens after routing decisions have been made, but before any local processing takes place.

3. NF_INET_FORWARD happens when a packet is to be forwarded through the system. When traffic was routed between networks (for example, from external to internal hosts), this hook was riggered. This confirmed that the system is forwarding packets rather than consuming them.

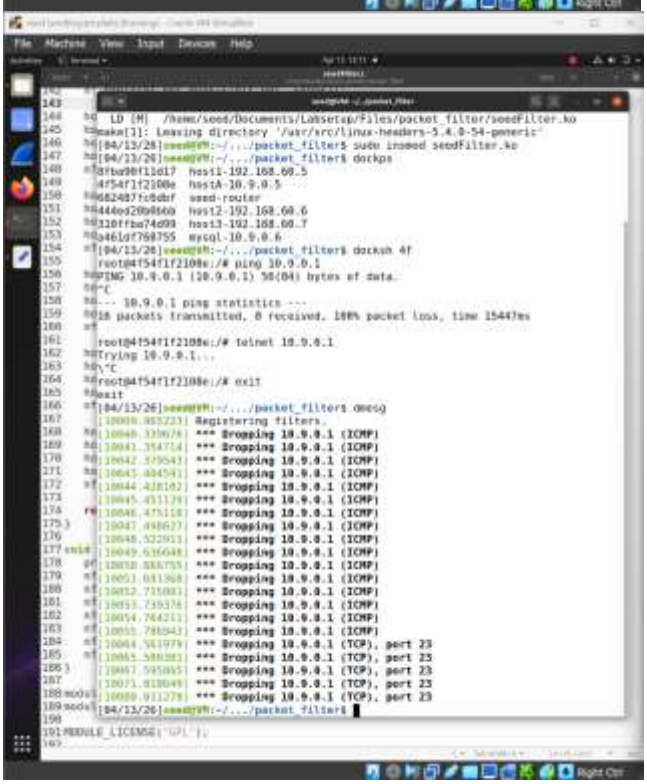
4. NF_INET_LOCAL_OUT happens when a locally generated packet is about to leave the system. When commands like dig @8.8.8.8 www.example.com were executed, logs showed activity at LOCAL_OUT. This makes sense because the packet originates from the local machine.

5. NF_INET_POST_ROUTING happens before a packet is sent out on a network interface. After LOCAL_OUT or FORWARD, packets reached POST_ROUTING just before being sent out. Logs confirmed that this is the last checkpoint before transmission.





```
hook2.hook = blockUDP;
hook2.hooknum = NF_INET_POST_ROUTING;
hook2.pf = PF_INET;
hook2.priority = NF_IP_PRI_FIRST;
nf_register_net_hook(&init_net, &hook2);

hook3.hook = printInfo;
hook3.hooknum = NF_INET_PRE_ROUTING;
hook3.pf = PF_INET;
hook3.priority = NF_IP_PRI_FIRST;
nf_register_net_hook(&init_net, &hook3);

hook4.hook = printInfo;
hook4.hooknum = NF_INET_LOCAL_IN;
hook4.pf = PF_INET;
hook4.priority = NF_IP_PRI_FIRST;
nf_register_net_hook(&init_net, &hook4);

hook5.hook = printInfo;
hook5.hooknum = NF_INET_FORWARD;
hook5.pf = PF_INET;
hook5.priority = NF_IP_PRI_FIRST;
nf_register_net_hook(&init_net, &hook5);

hook6.hook = blockPing;
hook6.hooknum = NF_INET_PRE_ROUTING;
hook6.pf = PF_INET;
hook6.priority = NF_IP_PRI_FIRST;
nf_register_net_hook(&init_net, &hook6);

hook7.hook = blockTelnet;
hook7.hooknum = NF_INET_PRE_ROUTING;
hook7.pf = PF_INET;
hook7.priority = NF_IP_PRI_FIRST;
nf_register_net_hook(&init_net, &hook7);

return 0;
}

void removeFilter(void) {
printk(KERN_INFO "The filters are being removed.\n");
nf_unregister_net_hook(&init_net, &hook1);
nf_unregister_net_hook(&init_net, &hook2);
nf_unregister_net_hook(&init_net, &hook3);
nf_unregister_net_hook(&init_net, &hook4);
nf_unregister_net_hook(&init_net, &hook5);
nf_unregister_net_hook(&init_net, &hook6);
nf_unregister_net_hook(&init_net, &hook7);
}
```



```
LD [M]  /home/seed/Documents/Labsetup/Files/packet_filter/seedFilter.ko
make[1]: Leaving directory '/usr/src/linux-headers-5.4.0-54-generic'
[04/13/26]seed@VM:~/.../packet_filter$ sudo insmod seedFilter.ko
[04/13/26]seed@VM:~/.../packet_filter$ dockps
8fba08f11d17  host1-192.168.60.5
4f54f1f2108e  hostA-10.9.0.5
6824837c8dbf  seed-router
444ed20b4b6b  host2-192.168.60.6
110ffba74d09  host3-192.168.60.7
b461df768755  mysql-10.9.0.6
[04/13/26]seed@VM:~/.../packet_filter$ docksh 4f
root@4f54f1f2108e:/# ping 10.9.0.1
PING 10.9.0.1 (10.9.0.1) 56(84) bytes of data.
^C
--- 10.9.0.1 ping statistics ---
10 packets transmitted, 0 received, 100% packet loss, time 15447ms

root@4f54f1f2108e:/# telnet 10.9.0.1
Trying 10.9.0.1...
^C
root@4f54f1f2108e:/# exit
exit
[04/13/26]seed@VM:~/.../packet_filter$ dmesg
[10040.483223] Registering filters.
[10040.339676] *** Dropping 10.9.0.1 (ICMP)
[10041.354714] *** Dropping 10.9.0.1 (ICMP)
[10042.379543] *** Dropping 10.9.0.1 (ICMP)
[10043.404541] *** Dropping 10.9.0.1 (ICMP)
[10044.428102] *** Dropping 10.9.0.1 (ICMP)
[10045.451129] *** Dropping 10.9.0.1 (ICMP)
[10046.475118] *** Dropping 10.9.0.1 (ICMP)
[10047.498627] *** Dropping 10.9.0.1 (ICMP)
[10048.522011] *** Dropping 10.9.0.1 (ICMP)
[10049.636648] *** Dropping 10.9.0.1 (ICMP)
[10050.684755] *** Dropping 10.9.0.1 (ICMP)
[10051.681368] *** Dropping 10.9.0.1 (ICMP)
[10052.715003] *** Dropping 10.9.0.1 (ICMP)
[10053.739376] *** Dropping 10.9.0.1 (ICMP)
[10054.764251] *** Dropping 10.9.0.1 (ICMP)
[10055.788043] *** Dropping 10.9.0.1 (ICMP)
[10064.561979] *** Dropping 10.9.0.1 (TCP), port 23
[10065.580081] *** Dropping 10.9.0.1 (TCP), port 23
[10067.595065] *** Dropping 10.9.0.1 (TCP), port 23
[10071.618049] *** Dropping 10.9.0.1 (TCP), port 23
[10080.011279] *** Dropping 10.9.0.1 (TCP), port 23
[04/13/26]seed@VM:~/.../packet_filter$
```

Create new filters for both ping and telnet involved modifying the existing blockUDP function and adding ICMP for ping and changing to TCP port 23 for telnet. After recompile and loading the module into the kernel, both ping and telnet are successfully blocked. Running dmesg confirms successful drop of both ICMP and Telnet

Task 2

Objective:

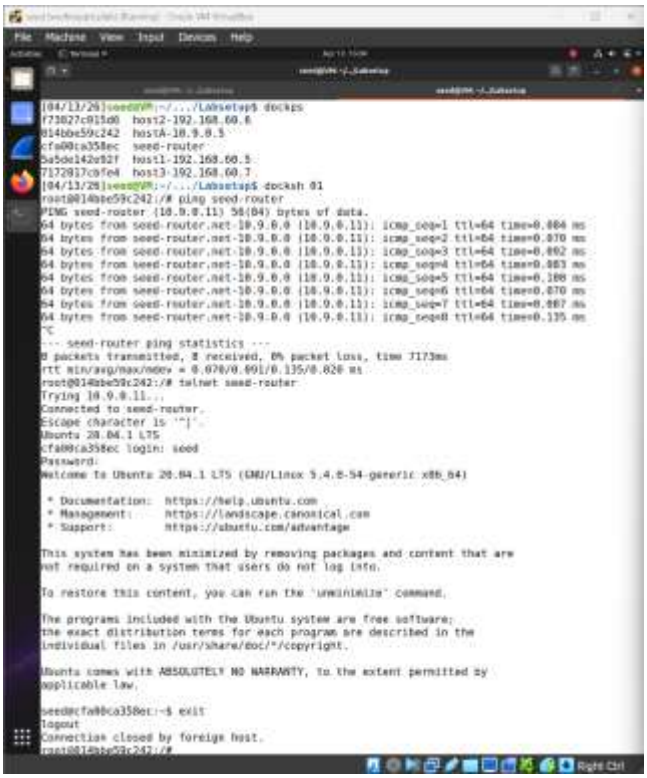
The objective is to master the use of the iptables utility to configure a built-in stateless packet-filtering firewall for a network. Success involves applying hierarchical rules to protect a router and an internal network from unauthorized access based on specific IP addresses and port numbers.

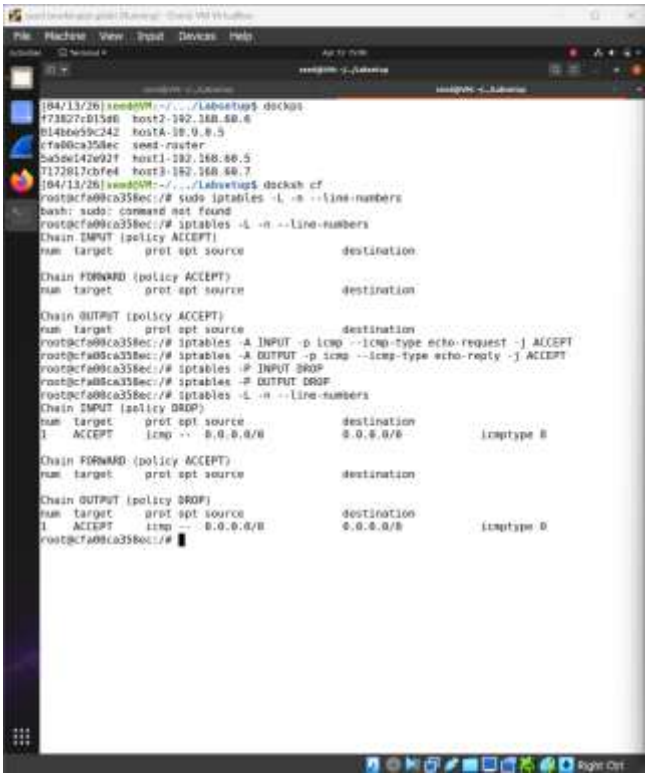
Task 2A

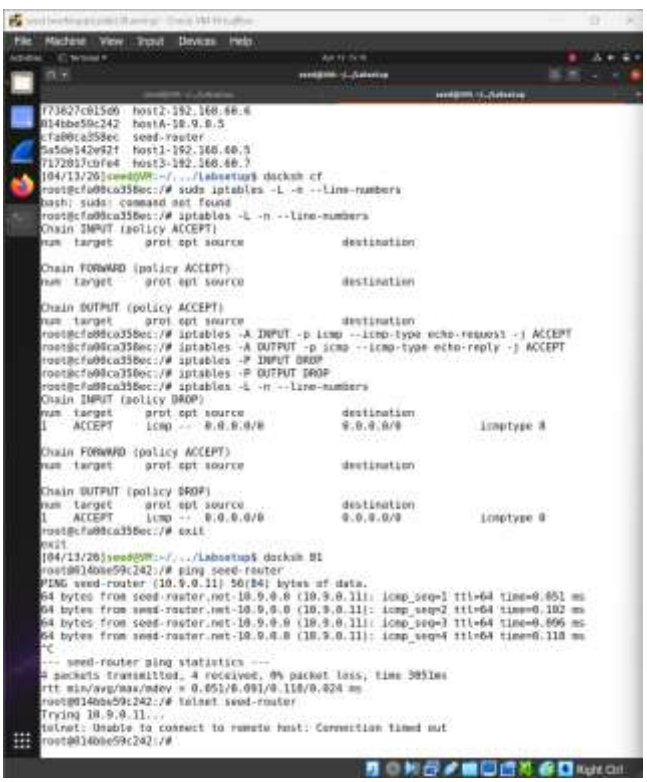
Instructions:

1. On the router container, allow ICMP traffic by running iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT and iptables -A OUTPUT -p icmp --icmp-type echo-reply -j ACCEPT.
2. Set default policies using iptables -P OUTPUT DROP and iptables -P INPUT DROP.
3. Test connectivity by running ping and telnet from another container, confirming ping works and telnet fails.
4. Restore the firewall state using iptables -F, iptables -P OUTPUT ACCEPT, and iptables -P INPUT ACCEPT.

Observations:







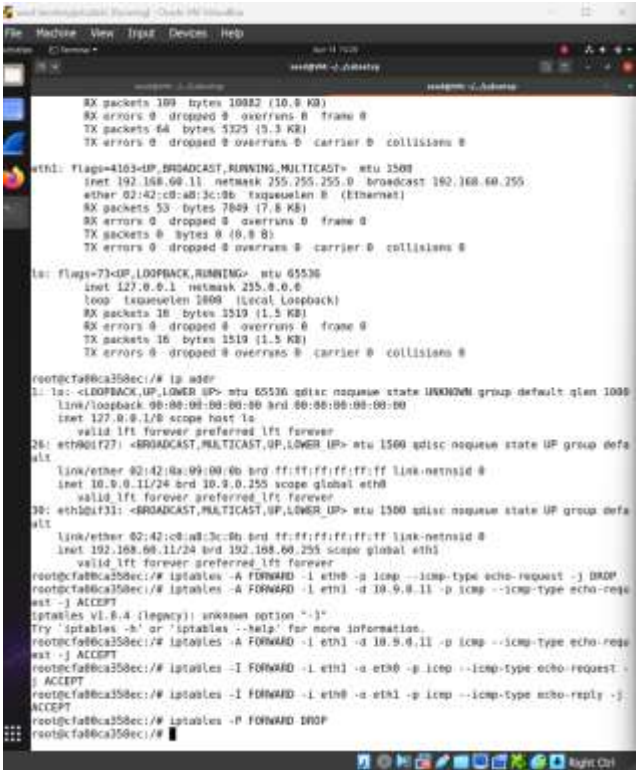
After successfully setting the correct configuration on the router's iptables to allow only ICMP, Telnet failed to connect as expected while ping worked between HostA and the router.

Task 2B

Instructions:

1. Identify network interfaces by running ip addr.
2. Add FORWARD rules using iptables -A FORWARD -p icmp ... to block external hosts from pinging internal hosts while allowing internal hosts to ping externally.
3. Verify functionality using ping tests between internal and external systems.
4. Clear rules using iptables -F before continuing.

Observations:



```
[04/13/26]seed@VM:~/.../Labsetup$ dockps
f73827c815d6  host2-192.168.60.6
014bbe59c242  hostA-10.9.0.5
cfa00ca358ec  seed-router
5a5de142e02f  host1-192.168.60.5
7172817cafe4  host3-192.168.60.7
[04/13/26]seed@VM:~/.../Labsetup$ docksh seed-router
root@cfa00ca358ec:/# iptables -L -n --line-numbers
Chain INPUT (policy ACCEPT)
num  target     prot opt source               destination

Chain FORWARD (policy DROP)
num  target     prot opt source               destination
1    ACCEPT     icmp --  0.0.0.0/0            0.0.0.0/0            icmptype 8
2    ACCEPT     icmp --  0.0.0.0/0            0.0.0.0/0            icmptype 8
3    DROP       icmp --  0.0.0.0/0            0.0.0.0/0            icmptype 8
4    ACCEPT     icmp --  0.0.0.0/0            10.9.0.11            icmptype 8

Chain OUTPUT (policy ACCEPT)
num  target     prot opt source               destination
root@cfa00ca358ec:/#
```

```
[04/13/26]seed@VM:~/.../Labsetup$ dockps
f73827c815d6  host2-192.168.60.6
014bbe59c242  hostA-10.9.0.5
cfa00ca358ec  seed-router
5a5de142e02f  host1-192.168.60.5
7172817cafe4  host3-192.168.60.7
[04/13/26]seed@VM:~/.../Labsetup$ docksh 01
root@014bbe59c242:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
^C
--- 192.168.60.5 ping statistics ---
12 packets transmitted, 0 received, 100% packet loss, time 11243ms

root@014bbe59c242:/# ping 10.9.0.11
PING 10.9.0.11 (10.9.0.11) 56(84) bytes of data.
64 bytes from 10.9.0.11: icmp_seq=1 ttl=64 time=0.063 ms
64 bytes from 10.9.0.11: icmp_seq=2 ttl=64 time=0.107 ms
64 bytes from 10.9.0.11: icmp_seq=3 ttl=64 time=0.106 ms
64 bytes from 10.9.0.11: icmp_seq=4 ttl=64 time=0.097 ms
^C
--- 10.9.0.11 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3067ms
rtt min/avg/max/mdev = 0.063/0.093/0.107/0.017 ms
root@014bbe59c242:/# telnet 192.168.60.5
Trying 192.168.60.5...
telnet: Unable to connect to remote host: Connection timed out
root@014bbe59c242:/# ping google.com
PING google.com (142.250.114.100) 56(84) bytes of data.
64 bytes from rr-in-f100.1e100.net (142.250.114.100): icmp_seq=1 ttl=102 time=19.6 ms
64 bytes from rr-in-f100.1e100.net (142.250.114.100): icmp_seq=2 ttl=102 time=29.1 ms
64 bytes from rr-in-f100.1e100.net (142.250.114.100): icmp_seq=3 ttl=102 time=25.6 ms
64 bytes from rr-in-f100.1e100.net (142.250.114.100): icmp_seq=4 ttl=102 time=22.9 ms
^C
--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 19.589/24.289/29.056/3.487 ms
root@014bbe59c242:/# exit
exit
[04/13/26]seed@VM:~/.../Labsetup$
```

After setting the correct configuration on the router's iptables the following were successfully observed:

1. HostA (external) was unable to ping or telnet into the internal IPs.
2. HostA (external) was still able to ping other external IPs.
3. Host1 (internal) was able to ping external networks, but unable to telnet external IPs.

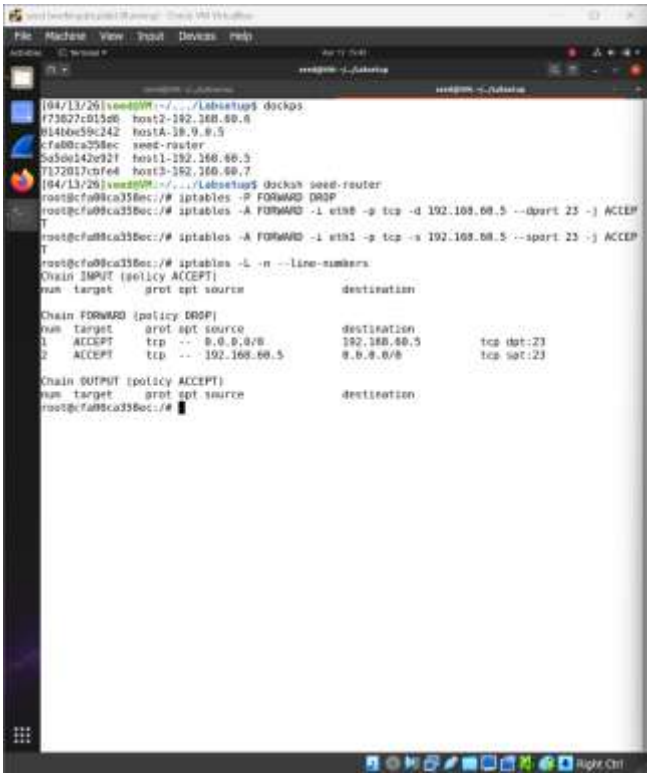
4. Host1 (internal) was able to ping other internal IPs.
5. The router is able to telnet external IPs.

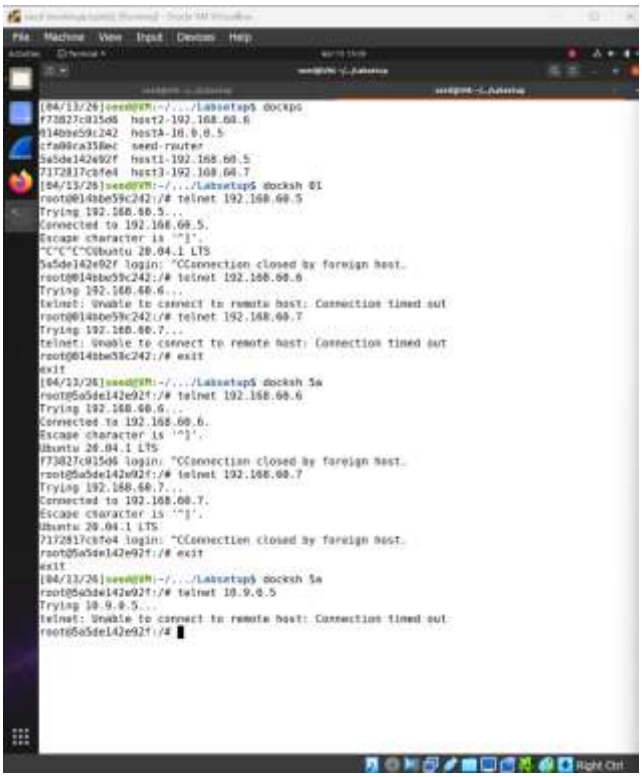
Task 2C

Instructions:

1. Configure FORWARD chain rules using iptables -A FORWARD -p tcp --dport 23 ... to allow external access only to 192.168.60.5.
2. Block access to other internal servers while allowing internal-to-internal communication and blocking internal-to-external traffic.
3. Test connectivity using telnet and confirm that only the allowed host is reachable.
4. Reset rules using iptables -F.

Observations:




File Machine View Input Devices Help
[04/13/26]seed@VM:~/.../Labsetup$ docksh 01
root@014bbe59c242:/# telnet 192.168.60.5
Trying 192.168.60.5...
Connected to 192.168.60.5.
Escape character is '^]'.
^C^C^C

```
[04/13/26]seed@VM:~/.../Labsetup$ dockps
f73827c815e6  host2-192.168.60.6
014bbe59c242  hostA-10.9.0.5
cfa80ca358ec  seed-router
5a5de142e92f  host1-192.168.60.5
7172817cafe4  host3-192.168.60.7
[04/13/26]seed@VM:~/.../Labsetup$ docksh 01
root@014bbe59c242:/# ping 192.168.60.5
PING 192.168.60.5 (192.168.60.5) 56(84) bytes of data.
64 bytes from 192.168.60.5: icmp_seq=1 ttl=63 time=0.130 ms
64 bytes from 192.168.60.5: icmp_seq=2 ttl=63 time=0.152 ms
64 bytes from 192.168.60.5: icmp_seq=3 ttl=63 time=0.066 ms
64 bytes from 192.168.60.5: icmp_seq=4 ttl=63 time=0.138 ms
^C
--- 192.168.60.5 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3074ms
rtt min/avg/max/mdev = 0.066/0.124/0.152/0.023 ms
root@014bbe59c242:/# nc -u 192.168.60.5 9090
hi
^C
root@014bbe59c242:/# nc  192.168.60.5 9090
hi tcp
^C
root@014bbe59c242:/#
```

```
[04/13/26]seed@VM:~/.../Labsetup$ docksh 5a
root@5a5de142e92f:/# nc -lu 9090
hi
^C
root@5a5de142e92f:/# nc -l 9090
nc: invalid option -- 'l'
usage: nc [-46CDdFhklNnrStUuvZz] [-I length] [-i interval] [-M ttl]
          [-m minttl] [-O length] [-P proxy_username] [-p source_port]
          [-q seconds] [-s source] [-T keyword] [-V rtable] [-W recvlimit] [-w timeout]
          [-X proxy_protocol] [-x proxy_address[:port]]          [destination] [port]
root@5a5de142e92f:/# nc -l 9090
hi tcp
root@5a5de142e92f:/#
```

Setting up Host1 to listen on port 9090 for TCP packets allowed Host1(internal) to show messages sent by HostA (external) over TCP port 9090. Running conntrac -L command on the router shows the communication between HostA and Host1.

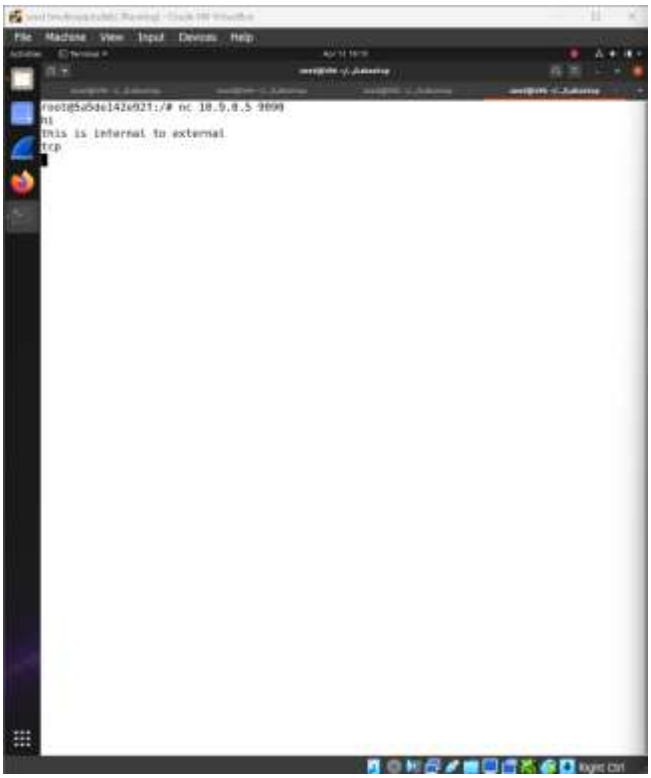
Task 3B

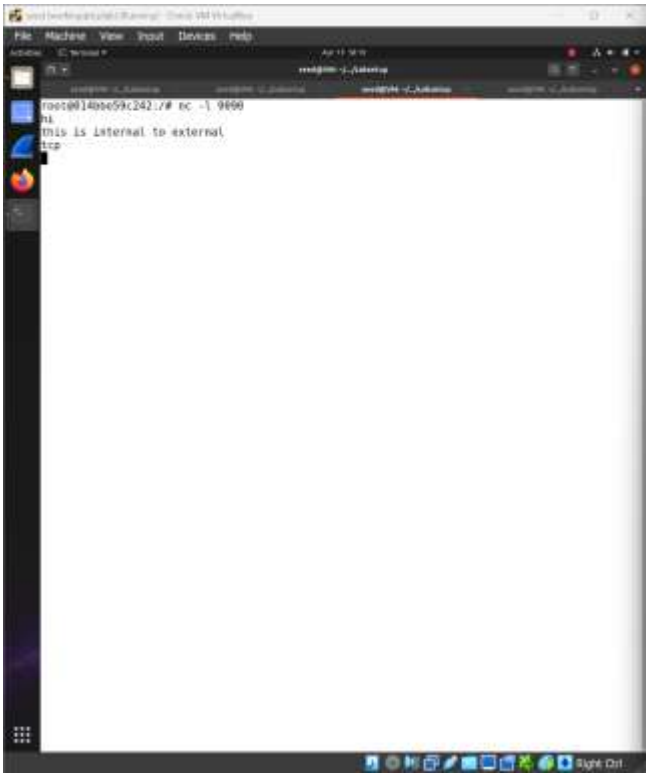
Instructions:

1. Allow established and related traffic using iptables -A FORWARD -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT.
2. Allow new TCP connections using iptables -A FORWARD -p tcp --syn -m conntrack --ctstate NEW -j ACCEPT.
3. Set the default policy to drop using iptables -P FORWARD DROP and modify previous rules to allow internal hosts to access external servers.
4. Verify connectivity and clear rules using iptables -F.

Observations:

(04/13/26)seed@VM:~/.../Labsetup$ docksh seed-router
root@cfa00ca358ec:/# iptables -P FORWARD DROP
root@cfa00ca358ec:/# iptables -A FORWARD -i eth0 -p tcp -d 192.168.60.5 --dport 23 -j ACCEP
T
root@cfa00ca358ec:/# iptables -A FORWARD -i eth1 -p tcp -s 192.168.60.5 --sport 23 -j ACCEP
T
root@cfa00ca358ec:/# iptables -A FORWARD -p tcp -m conntrack --cstate ESTABLISHED,RELATED -
j ACCEPT
iptables v1.8.4 (legacy): Couldn't load match `conntrack':No such file or directory
Try `iptables -h' or 'iptables --help' for more information.
root@cfa00ca358ec:/# iptables -A FORWARD -p tcp -m conntrack --cstate ESTABLISHED,RELATED -
j ACCEPT
iptables v1.8.4 (legacy): unknown option "--cstate"
Try `iptables -h' or 'iptables --help' for more information.
root@cfa00ca358ec:/# iptables -A FORWARD -p tcp -m conntrack --ctstate ESTABLISHED,RELATED
-j ACCEPT
root@cfa00ca358ec:/# iptables -A FORWARD -i eth1 -p tcp --syn -m conntrack --ctstate NEW -j
ACCEPT
root@cfa00ca358ec:/# iptables -L -n --line-numbers
Chain INPUT (policy ACCEPT)
num target prot opt source destination
Chain FORWARD (policy DROP)
num target prot opt source destination
1 ACCEPT tcp -- 0.0.0.0/0 192.168.60.5 tcp spt:23
2 ACCEPT tcp -- 192.168.60.5 0.0.0.0/0 tcp spt:23
3 ACCEPT tcp -- 0.0.0.0/0 0.0.0.0/0 ctstate RELATED,ESTABLIS
HED
4 ACCEPT tcp -- 0.0.0.0/0 0.0.0.0/0 tcp flags:0x17/0x02 ctst
ate NEW
Chain OUTPUT (policy ACCEPT)
num target prot opt source destination
root@cfa00ca358ec:/#


root@5a6de142e921:/# nc 10.9.0.5 9090
hi
This is internal to external
tcp



Similar to the previous subtask, except this time established and related connections are allowed, this is successfully shown through the communication between HostA (external) and Host1 (internal).

The stateless approach, used in the first set of rules, is simpler to implement and requires less computational overhead because the firewall examines each packet in isolation without maintaining a record of the connection state. However, this method is significantly less secure and more cumbersome because it requires manually opening holes for every possible return path and lacks the context to distinguish between legitimate response packets and unsolicited incoming traffic. In contrast, the stateful approach using connection tracking allows the firewall to automatically permit return traffic for established sessions, greatly reducing the number of rules required and simplifying management. The primary disadvantage of the stateful method is the increased memory and CPU usage needed to maintain the connection tracking table, which can potentially become a target for resource-exhaustion attacks. Despite this, the stateful approach is generally superior because it provides a much more granular level of security by ensuring that only packets belonging to a verified, bi-directional communication flow are allowed to pass through the router.

Task 4

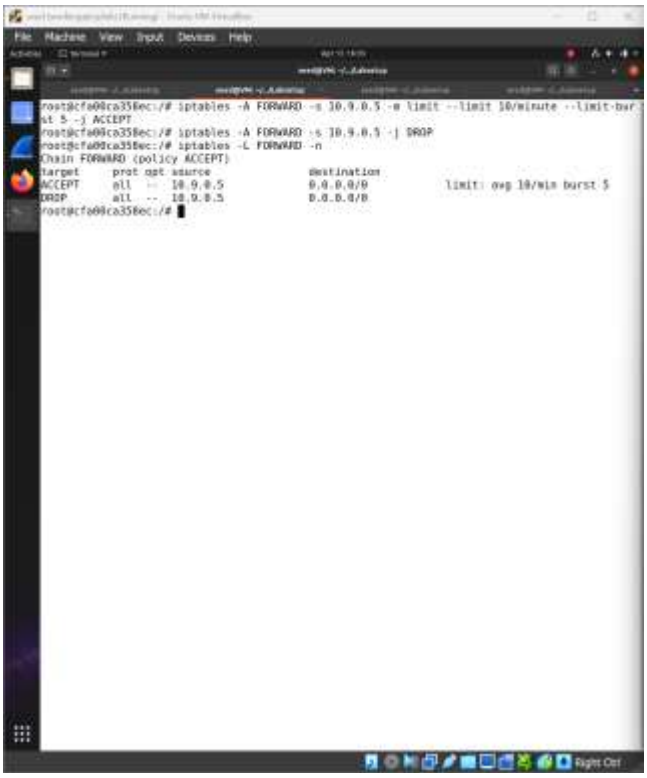
Objective:

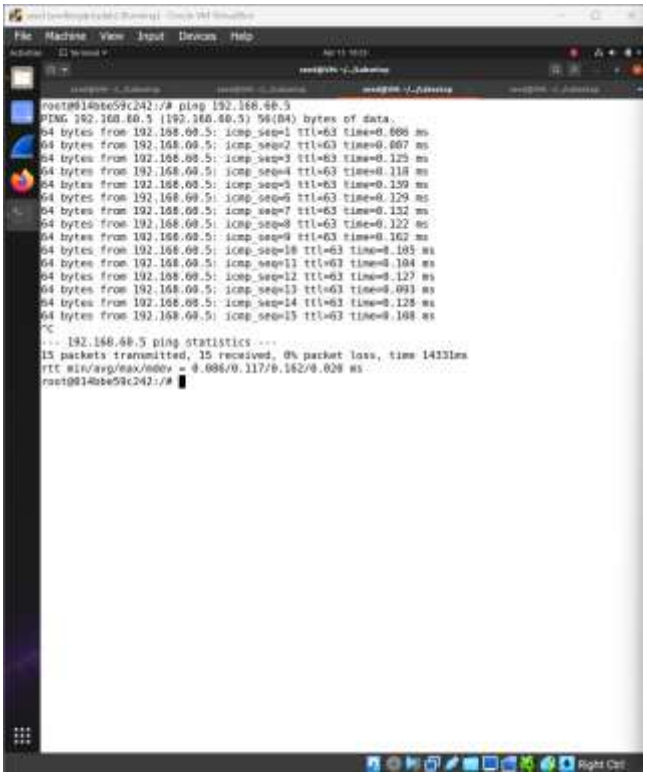
The objective is to utilize the limit module of iptables to control the volume of traffic permitted through the firewall. This provides a practical application for mitigating network congestion by restricting the average packet rate and defining burst thresholds.

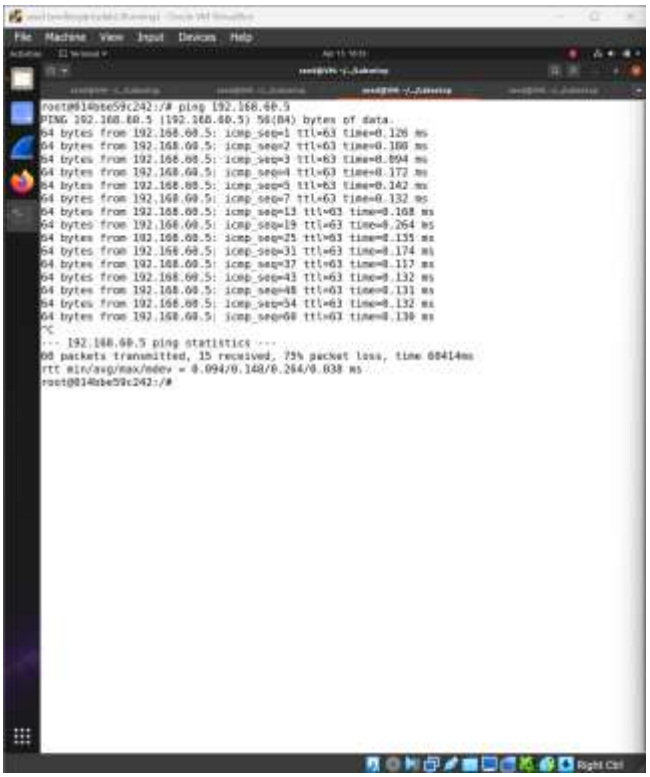
Instructions:

1. Add a rate-limiting rule using iptables -A FORWARD -s 10.9.0.5 -m limit --limit 10/minute --limit-burst 5 -j ACCEPT.
2. Add a drop rule using iptables -A FORWARD -s 10.9.0.5 -j DROP.
3. Test behavior by running ping 192.168.60.5 and compare results with and without the drop rule.

Observations:







The ICMP rate limit is implemented on the router, however, without the drop rule, the effects are not clear as the pings all go through. However, with the drop rule implemented, the rate limit effect can clearly bee seen as the packet loss is 75% up from 0%. This is because, prior to the drop rule, the router didn't know how to deal with the pings that exceeded rate limit. With the drop rule, the router knows to drop any pings that exceed the rate limit, leading to packet loss.

Task 5

Objective:

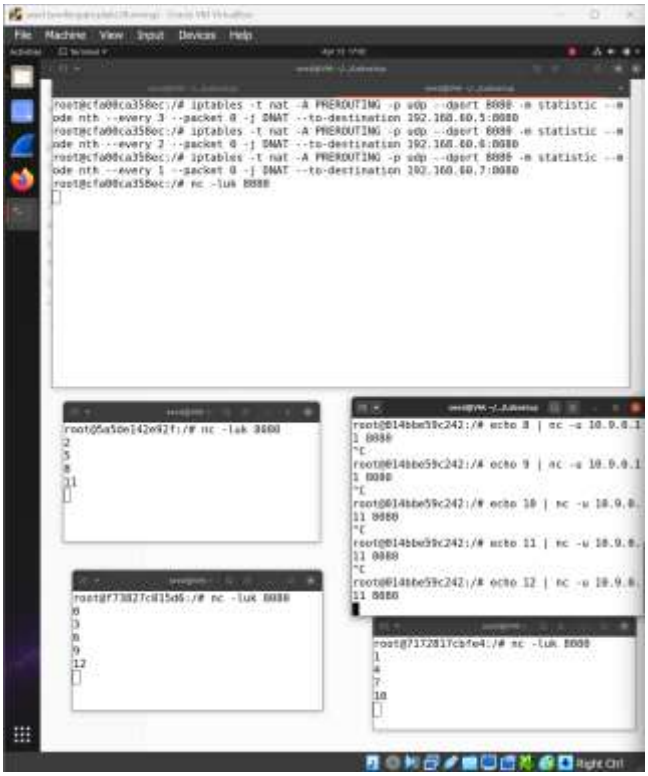
The objective is to apply the statistic module of iptables to distribute incoming network traffic across multiple internal servers. By using both nth (round-robin) and random (probability) modes, the task demonstrates how a firewall can function as a traffic orchestrator for optimized resource utilization.

Instructions:

1. Start UDP servers on internal hosts using nc -luk 8080.
2. Configure load balancing using iptables -t nat -A PREROUTING -p udp --dport 8080 -m statistic --mode nth --every 3 --packet 0 -j DNAT --to-destination 192.168.60.5:8080.
3. Add similar rules for hosts .6 and .7, then test distribution using echo hello | nc -u 10.9.0.11 8080.
4. Configure random load balancing using iptables -t nat -A PREROUTING -p udp --dport 8080 -m statistic --mode random --probability P -j DNAT --to-destination 192.168.60.5:8080.

5. Adjust probabilities for each host and test to verify roughly equal traffic distribution.

Observations:



Implementing the round robin (nth) load balancing shows how the router is able to effectively distribute the packets evenly among the 3 Hosts even at small packet loads, in this case 12 packets. Switching to random load balancing using percentage chance of receiving a packet, the “balance” is harder to see at such a small packet load. However, with increased

packet load, the random load balancing should be able to even distribute the packets based on the percentage.

Conclusion:

The exploration of firewall technologies confirms that both custom kernel-level modules and user-space utilities like iptables offer robust methods for securing a network. It was observed that while stateless firewalls are efficient for simple filtering, stateful inspection is required to handle complex protocols like TCP and UDP effectively. The implementation of rate limiting and load balancing further proves that firewalls are versatile tools capable of managing network performance alongside security. However, the risk of system instability during kernel module development highlights the necessity of precise hook management and unregistration. In summary, mastering these tools provides the necessary expertise to design resilient network architectures that protect sensitive internal resources.